Rapport technique : projet domotique STM32

Projet de domotique embarquée. Zakaria Boschetti , Léonard de Bibikoff—Sangngoen, Grp3 octobre 2026



1. Présentation du projet et architecture

Pour ce projet on a simulé une maison connectée autour d'une carte **STM32** .Elle a deux modes, HOME et AWAY, cinq capteurs : le **PIR** (détecte les mouvements), le **LDR** (mesure la luminosité), le **MQ-2** (détecte le gaz), le **Steam Sensor** (détecte la pluie) et l'**AHT20** (mesure la température et l'humidité) et plusieurs actionneurs : des **LEDs** (signalisation des états et des alarmes), un **buzzer** (sirène et bips), un

servomoteur (ouvre et ferme la fenêtre) et un écran **LCD** 16×2 en I²C (affichage local). La supervision se fait depuis le PC par l'UART.

Le fonctionnement est toujours le même : le Timer déclenche une acquisition, on lit les capteurs, on décide de ce qu'il faut faire, puis on met à jour les actionneurs, le **LCD** et l'UART.

Pour l'architecture, on a fait quelques choix. D'abord, les interruptions restent courtes : le **Timer 2** se contente de lever un drapeau (`AcqFlag`) toutes les 2 s, et tout le reste (mesures, driver I²C, écran) est traité dans la boucle `main`. Ensuite, on voulait des alarmes non bloquantes, donc pas de `HAL_Delay` : le clignotement des **LED** et le son sont gérés par les interruptions des timers.

Le **LCD** n'est jamais écrit dans une interruption, parce qu'il partage le bus I²C avec l'**AHT20** et que chaque écriture est bloquante. Les interruptions lèvent seulement `LcdFlag` et c'est le `main` qui rafraîchit l'écran. Ces deux drapeaux (`AcqFlag` et `LcdFlag`) sont déclarés `volatile`, puisqu'ils sont modifiés dans une interruption et lus dans le `main`.

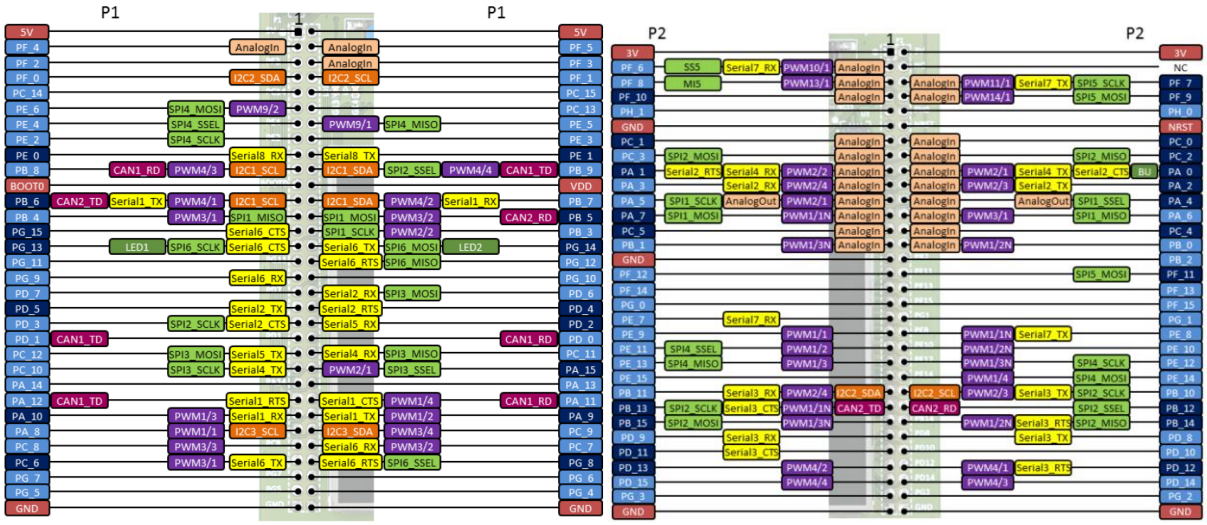
Enfin, chaque capteur à seuil a une hystérésis (deux seuils), pour que le système ne bascule pas en boucle quand la mesure est autour de la limite.

Horloges (SystemClock_Config)

On utilise l'oscillateur interne HSI (16 MHz) et la PLL pour monter à 180 MHz. Le tableau donne la config qu'on a retenue.

Élément	Valeur	Calcul
Source	HSI 16 MHz	oscillateur interne
PLL	180 MHz	16 MHz / 8 × 180 / 2
HCLK (SYSCLK)	180 MHz	AHB diviseur 1
APB1	45 MHz (timers : **90 MHz**)	HCLK / 4, les timers sont ×2
APB2	90 MHz (timers : **180 MHz**)	HCLK / 2, les timers sont ×2

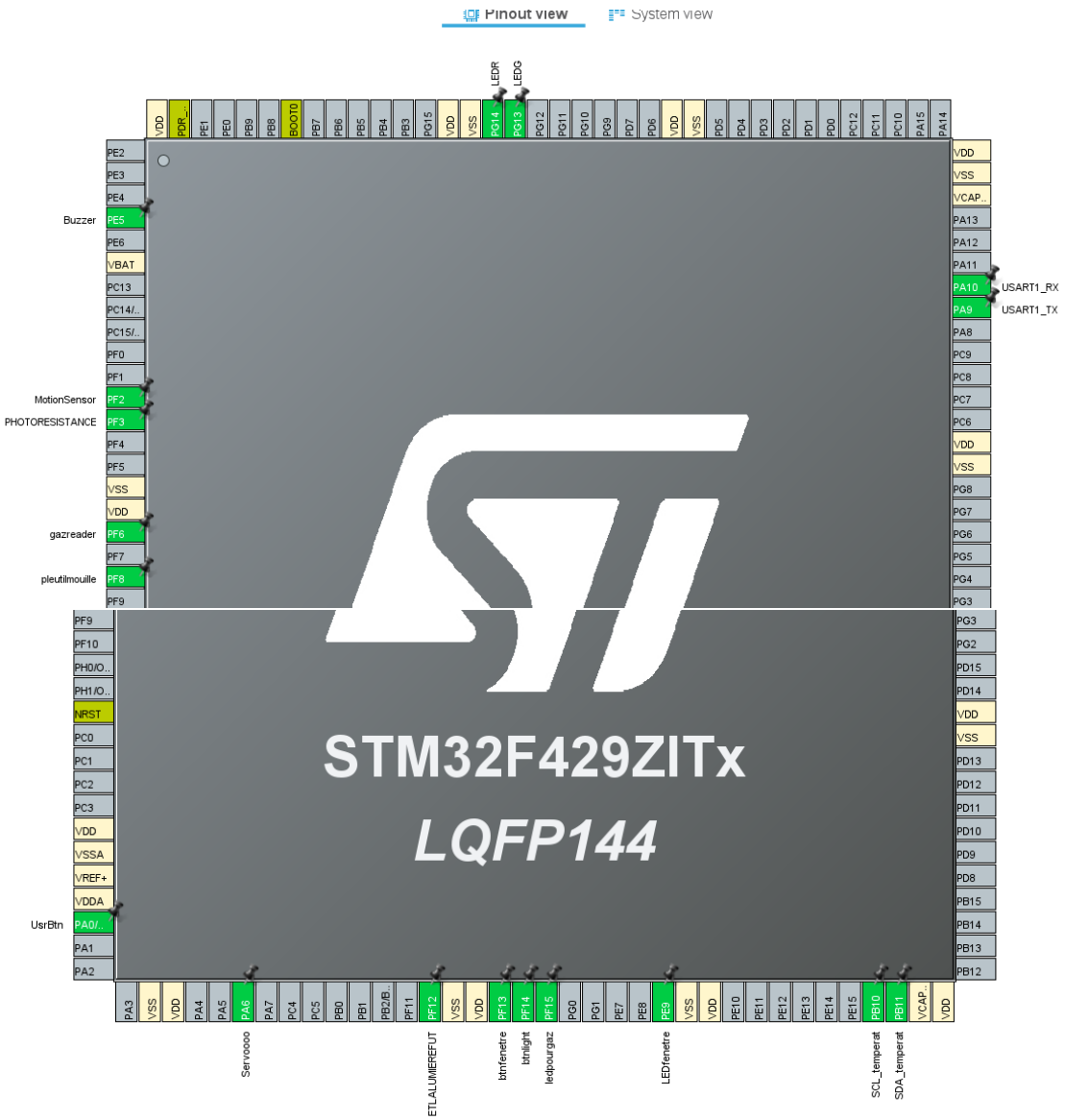
2. Broches utilisées



Fonction	Broche	Périphérique	Détail / justification
UART supervision	PA9 (TX), PA10 (RX)	USART1	115200 bauds, 8N1, vers le PC via ST-LINK
Bouton MODE (USER) bascule HOME / AWAY	PA0	EXTI0	front montant, bouton intégré à la carte
PIR détection de mouvement	PF2	EXTI2	front montant (début de détection)
Buzzer passif sirène et bips	PE5	TIM9_CH1 (PWM)	PWM : la fréquence = la note jouée
LED validation timer / mode AWAY	PG13 (LD3)	GPIO	LED de la carte
LED alarme intrusion	PG14 (LD4)	GPIO	LED de la carte
LDR mesure de luminosité	PA3	ADC1_IN3	entrée ADC
LED éclairage	PF12	GPIO	sortie
Bouton Lumière éclairage manuel	PF14	EXTI14	front montant, pull-down
MQ-2 détection de gaz	PA5	ADC1_IN5	entrée ADC
LED alarme gaz	PF15	GPIO	sortie

Fonction	Broche	Périphérique	Détail / justification
Servomoteur ouverture de la fenêtre	PA6	TIM3_CH1 (PWM, AF2)	50 Hz
Steam Sensor détection de pluie	PA7	ADC1_IN7	entrée ADC
Bouton Fenêtre fenêtre manuelle	PF13	EXTI13	front montant, pull-down
LED fenêtre allumée = ouverte	PE9	GPIO	sortie
I²C SCL horloge du bus	PB10	I2C2 (AF4)	open-drain, pull-up
I²C SDA données du bus	PB11	I2C2 (AF4)	open-drain, pull-up

Côté alimentation, l'**AHT20** et le **LCD** sont en 5 V. Le **MQ-2**, le **servomoteur** et tous les autres capteurs et boutons sont en 3.3 V. Il faut une masse commune entre la carte, le **servomoteur** et les capteurs.



Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull...	Maximum output ...	User Label	Modified
PA0/WKUP	n/a	n/a	External Interrupt...	No pull-up and n...	n/a	UsrBtn	☑
PE9	n/a	Low	Output Push Pull	No pull-up and n...	Low	LEDfenetre	☑
PF2	n/a	n/a	External Interrupt...	No pull-up and n...	n/a	MotionSensor	☑
PF12	n/a	Low	Output Push Pull	No pull-up and n...	Low	ETLALUMIERE...	☑
PF13	n/a	n/a	External Interrupt...	No pull-up and n...	n/a	btnfenetre	☑
PF14	n/a	n/a	External Interrupt...	No pull-up and n...	n/a	btnlight	☑
PF15	n/a	Low	Output Push Pull	No pull-up and n...	Low	ledpourgaz	☑
PG13	n/a	Low	Output Push Pull	No pull-up and n...	Low	LEDG	☑
PG14	n/a	Low	Output Push Pull	No pull-up and n...	Low	LEDR	☑

3. Tâche périodique : Timer 2

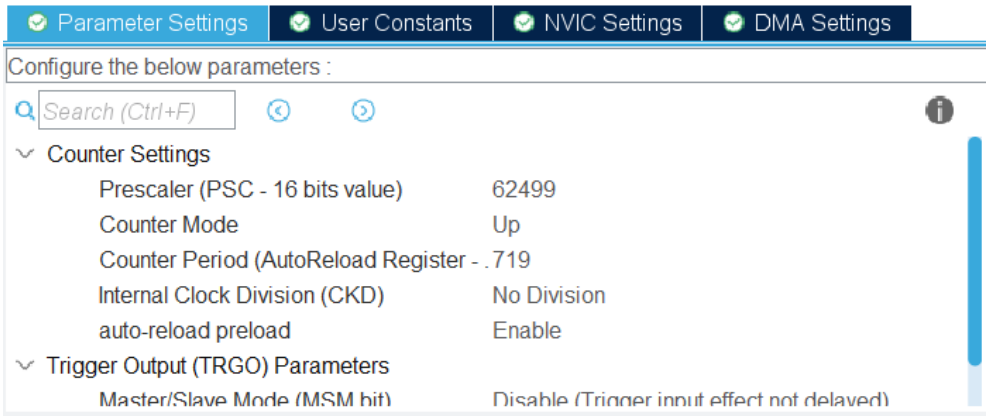
Le **Timer 2**, qui cadence toute l'acquisition, est sur l'APB1, donc son horloge est de 90 MHz. Avec un prescaler de 62499 on obtient 90 MHz / 62500 = 1440 Hz, puis avec une période (ARR) de 719 on arrive

à 1440 / 720 = 2 Hz. Le timer compte vers le haut et l'interruption de mise à jour est activée : on a donc une interruption toutes les 0,5 s. L'acquisition des capteurs se fait toutes les 4 interruptions, soit toutes les 2 s.

On a pris 2 s car la consigne impose entre 1 et 5 s. C'est assez rapide pour réagir au gaz ou à la pluie, et assez lent pour laisser le temps à l'**AHT20** de faire sa conversion (80 ms) et à l'écran et au **LCD** de se mettre à jour (voir scénario 7).

Le tick de 0,5 s sert aussi de base de temps pour les clignotements (**LED** d'alarme, bips du gaz), ce qui nous évite un deuxième timer. Le prescaler et l'ARR sont tous les deux inférieurs à 65536, donc ça rentre dans un compteur 16 bits (2^16 = 65536).

Pour vérifier, en mode AWAY la **LED** PG13 change d'état à chaque tick. À l'oscilloscope sur PG13, on doit donc relever une période de 1,000 s (basculement toutes les 0,5 s).



4. Supervision : UART

La supervision passe par l'**USART1** (communication série avec le PC) (PA9 en TX, PA10 en RX), reliée au **ST-LINK** (qui fait le lien avec le PC) : sur le PC on la retrouve comme un port série virtuel. On l'a réglée à 115200 bauds, 8 bits, sans parité, 1 bit de stop, sans contrôle de flux, avec un sur-échantillonnage ×16. C'est un débit standard et nos messages sont courts (environ 87 µs par caractère), donc le temps d'envoi est négligeable.

Dans le terminal on a défini une zone de défilement : les deux premières lignes affichent l'état (pour imiter un écran 16×2) et les messages de diagnostic défilent en dessous.

On y envoie les mesures (**LDR**=… MQ2=… RAIN=…, TEMP=…C HUM=…%RH), le mode, les alarmes (GAS ALERT!) et l'état de l'éclairage (LIGHT ON (auto)) et de la fenêtre (WINDOW OPEN (auto)). Ces infos sont ensuite reprises sur le **LCD**.

Basic Parameters	
Baud Rate	115200 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1
Advanced Parameters	
Data Direction	Receive and Transmit
Over Sampling	16 Samples

5. Scénario 2 : modes HOME / AWAY et intrusion

Le **bouton MODE** est le **bouton USER** de la carte (PA0), qui sert à passer de HOME à AWAY, en EXTI sur front montant. Quand on appuie, il bascule `IsAway`, remet l'alarme à zéro, éteint les **LED** et met à jour l'affichage (HOME ou AWAY). Au démarrage on est en HOME : passer en AWAY revient à mettre l'alarme, et repasser en HOME à la retirer.

Le **PIR** (PF2, EXTI sur front montant), qui détecte les mouvements, déclenche l'interruption au début d'une détection. En HOME on ignore la détection, en AWAY elle lance l'alarme d'intrusion (`AlarmOn` = 1).

L'alarme d'intrusion fait clignoter la **LED** rouge PG14 (toggle toutes les 0,5 s dans le **Timer 2**) et joue une sirène sur le **buzzer**, dont la fréquence balaie environ 1 à 2 kHz à 4 Hz. Tout ça est non bloquant : la sirène est mise à jour à chaque période PWM, dans l'interruption de **TIM9**.

Pour le **buzzer** (sirène et bips), **TIM9** tourne à 180 MHz / 625 = 288 kHz. La fréquence du son vaut 288000 / période, avec un rapport cyclique de 50 %. On n'a qu'un seul **buzzer** pour le gaz, l'intrusion et les bips de mode, donc on joue sur la fréquence pour avoir un son différent selon l'alarme.

Pour le **bouton USER**, on n'a pas mis d'anti-rebond : il est traité en EXTI sans filtrage. Un rebond peut donc provoquer un double basculement, ce qui peut causer des erreurs comme on l'a vu en TP. Il serait judicieux d'ajouter un filtre anti-rebond.

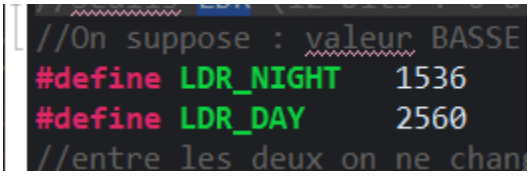
6. Scénario 3 : éclairage automatique et manuel

Les trois capteurs analogiques (**LDR**, **MQ-2**, **Steam Sensor**) utilisent le même **ADC**. On l'a réglé en 12 bits (0 à 4095), avec une conversion unique lancée par logiciel et un alignement à droite. L'horloge de l'**ADC** est de 90 MHz / 4 = 22,5 MHz (le maximum est 36 MHz).

Pour le temps d'échantillonnage on a pris 56 cycles : nos capteurs sont des ponts diviseurs d'impédance élevée, donc un temps long donne une mesure plus stable. Le temps de conversion total est de (56 + 12) / 22,5 MHz ≈ 3 µs. Une seule fonction `ReadADC(canal)` configure le canal à chaque appel.

Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-do...	Maximum output spe..	User Label	Modified
PF3	ADC3_IN9	n/a	Analog mode	No pull-up and no pull..	n/a	PHOTORESISTAN...	☑
PF6	ADC3_IN4	n/a	Analog mode	No pull-up and no pull..	n/a	gazreader	☑
PF8	ADC3_IN6	n/a	Analog mode	No pull-up and no pull..	n/a	pleutilmouille	☑

Pour l'éclairage automatique, la valeur du **LDR** (capteur de luminosité) est comparée à deux seuils (hystérésis) : c'est la nuit si la valeur est inférieure à `LDR_NIGHT`, et le jour si elle dépasse `LDR_DAY`. Entre les deux, l'état ne change pas.



En mode AWAY, l'éclairage reste éteint quelle que soit la luminosité.

Le **bouton Lumière** (PF14), commande manuelle de l'éclairage, bascule la **LED** (PF12) et passe en mode manuel. Ce mode dure jusqu'au prochain passage jour/nuit, puis l'automatique reprend. En AWAY, le bouton est ignoré.

Pour éviter les rebonds on a mis un filtre logiciel de 200 ms (avec `HAL_GetTick`). On l'a choisi pour sa simplicité : il ne bloque pas le CPU et on peut faire pareil sur les autres boutons. L'EXTI est en front montant avec une pull-down interne, pour qu'une entrée flottante ne déclenche pas de faux appuis.

On a relevé les valeurs de l'**ADC** sur l'UART dans plusieurs conditions :

Condition	Valeur ADC mesurée
Pleine lumière	4000
Pièce éclairée normalement	3000
Ombre / main sur le capteur	2700
Obscurité	900-1530

7. Scénario 4 : détection de gaz (MQ-2)

Le **MQ-2**, capteur de gaz, est lu sur l'**ADC1** canal 5 (PA5), toutes les 2 s et en permanence (en HOME comme en AWAY), puisque c'est une fonction de sécurité.

On a remarqué que le capteur est inversé : plus il y a de gaz, plus la valeur diminue. L'alarme se déclenche donc quand la valeur passe sous `GAS_ON` (1400) et elle ne s'arrête que quand la valeur repasse au-dessus de `GAS_OFF` (1600), c'est-à-dire quand l'air est vraiment revenu à la normale.

En alarme, la **LED** gaz (PF15) clignote et le **buzzer** fait des bips à 1500 Hz, les deux pilotés par le **Timer 2** pour rester non bloquants. Le message GAS ALERT est envoyé sur l'UART et affiché sur le **LCD**.

Le gaz est prioritaire sur l'intrusion pour le **buzzer** et pour l'écran. Quand l'alarme gaz se termine, la sirène d'intrusion reprend si elle était active.

8. Scénario 5 : fenêtre motorisée

8.1 Servomoteur : PWM

Le **servomoteur**, qui ouvre et ferme la fenêtre, est piloté par **TIM3** canal 1 sur PA6 (AF2), avec une horloge timer de 90 MHz. Le prescaler vaut 89, donc 90 MHz / 90 = 1 MHz et 1 tick = 1 µs. La période (ARR) est de 19999, soit 20000 µs = 20 ms = 50 Hz, qui est la fréquence standard d'un **servomoteur**. Le registre de comparaison correspond directement à la largeur d'impulsion en µs (1000 = 1 ms, 2000 = 2 ms), ce qui rend le réglage et la caractérisation plus faciles.

Positions utilisées :

```
#define SERVO_CLOSED    1000
#define SERVO_OPEN      2000
```

8.2 Bouton Fenêtre

Le **bouton Fenêtre** (ouverture et fermeture manuelles) est sur PF13 en EXTI, avec le même anti-rebond logiciel de 200 ms que pour la lumière.

8.3 Détection de pluie (Steam Sensor)

Le **Steam Sensor**, capteur de pluie, est lu sur l'**ADC1** canal 7 (PA7), toutes les 2 s. Sa tension augmente avec l'humidité. On a mis une hystérésis : il pleut si la valeur dépasse `RAIN_ON` (1500) et la pluie est finie quand elle repasse sous `RAIN_OFF` (1000).

Quand la pluie est détectée, la fenêtre se ferme et reste fermée tant qu'il pleut. Si on appuie sur le bouton pendant ce temps, il est refusé et un message est envoyé sur l'UART.

```
#define RAIN_ON      1500    //au dessus : il pleut
#define RAIN_OFF     1000    //en dessous : la pluie est finie
```

8.4 Règle de décision de la fenêtre (priorités)

Plusieurs choses peuvent commander la fenêtre, donc on a défini un ordre de priorité :

Priorité	Condition	Résultat
1	Mode AWAY	Fermée
2	Pluie détectée	Fermée
3	Action manuelle (bouton)	Selon la demande de l'utilisateur
4	Ventilation automatique (AHT20)	Ouverte si trop chaud ou trop humide

L'état de la fenêtre est affiché sur l'UART (WINDOW OPEN / WINDOW CLOSED, avec la raison : auto, button, rain ou away) et sur le **LCD** (W:OPN / W:CLS).

9. Scénario 6 : AHT20 (I²C)

9.1 Configuration de l'I²C

L'**AHT20**, capteur de température et d'humidité, est sur le bus **I2C2** : SCL = PB10 et SDA = PB11 (AF4), en open-drain avec pull-up.

Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-do...	Maximum output spe...	User Label
PB10	I2C2_SCL	n/a	Alternate Function O...	No pull-up and no pull...	Very High	SCL_temperat
PB11	I2C2_SDA	n/a	Alternate Function O...	No pull-up and no pull...	Very High	SDA_temperat

Son adresse est 0x38 sur 7 bits (0x70 en écriture, 0x71 en lecture). Dans le code on écrit `0x38<<1`, car la HAL attend l'adresse décalée d'un bit.

Petite remarque : sur la carte Discovery, PB10 et PB11 sont aussi reliées à l'écran **LCD** de la carte (LTDC). Comme ce périphérique n'est pas initialisé, il n'y a pas de conflit.

9.2 Driver

```
HAL_I2C_Master_Transmit (&hi2c2, SLAVE_ADDRESS_LCD, (uint8_t *) data_t, 4, 100);   // etait &hi2c1
```

On a changé l'adresse car `hi2c1` n'était pas celui qu'on utilisait.

9.3 Ouverture de la fenêtre

Condition	Seuil	Justification
Ouverture	température > **26,0 °C** ou humidité > **70 %RH**	au-dessus de la zone de confort (20 à 26 °C), et au-delà de 70 % le risque de moisissures augmente
Fermeture	température < **24,0 °C** et humidité < **60 %RH**	fermeture seulement quand les **deux** sont revenues à la normale

On a mis une hystérésis de 2 °C et de 10 %RH, pour que la fenêtre ne s'ouvre et ne se ferme pas en boucle autour du seuil.

Le bouton inverse l'état réel de la fenêtre et passe en mode manuel. Cet état dure jusqu'au prochain changement des conditions de ventilation, puis l'automatique reprend la main. La pluie et le mode AWAY sont plus prioritaires que l'automatique et que le bouton (voir 8.4).

Une limite : on n'a qu'un seul capteur, à l'intérieur, donc le système ne sait pas s'il fait plus frais dehors.

10. Affichage local : LCD 1602 I²C

Pour le **LCD** (affichage local de l'état du système) on est partis de la bibliothèque fournie (`i2c-lcd.c` / `i2c-lcd.h`), qu'on a adaptée en remplaçant `hi2c1` par `hi2c2`, qui est notre bus. L'adresse du **LCD** est 0x27. Il est sur le même bus que l'**AHT20** (0x27 et 0x38), donc pas de conflit.

Au démarrage on teste la présence du **LCD** avec `HAL_I2C_IsDeviceReady`. S'il est absent, on l'ignore pour ne pas ralentir le système et l'UART affiche **LCD** not found.

Le main rafraîchit l'écran après chaque acquisition et à chaque changement d'état (mode, alarme, lumière, fenêtre). On n'utilise pas `lcd_clear()` : chaque ligne est réécrite avec exactement 16 caractères, ce qui évite le scintillement. On a quand même parfois des soucis de rafraîchissement quand on reboot/redémarre le programme.

Voici ce que le **LCD** affiche selon la situation :

Situation	Ligne 1	Ligne 2
Normal	`HOME L:ON  W:OPN`	`T:24.5C H:45.2%`
Gaz (prioritaire)	`** GAS ALERT **`	`Air quality bad`
Intrusion	`!! INTRUSION !!`	`Alarm active`
Pluie	mode, lumière, fenêtre	`RAIN - W closed`
Erreur AHT20	mode, lumière, fenêtre	`AHT20 ERROR`

11. Gestion des conflits et priorités globales

Comme plusieurs fonctions se partagent le **buzzer** ou **l'écran** ou **la fenêtre** ou **l'éclairage**, on a fixé des priorités :

Ressource	Règle
Buzzer	gaz > intrusion > bips
Écran	gaz > intrusion > normal
Fenêtre	AWAY > pluie > bouton > auto
Éclairage	AWAY > bouton > auto
Bus I²C	Accès uniquement dans le `main`, jamais en interruption